

Chat to SQL: Data Jobs Q&A - Project Documentation

Oct 5, 2026 · @Rajat Mehta

A Streamlit app that lets anyone ask plain-English questions about 786,000 data-job postings and get back a table, without writing SQL. Gemini Flash Lite turns each question into a DuckDB query; the app checks the query is read-only, runs it, and shows the SQL, the result table and an optional chart.

Summary of approach

- **Data:** the public Hugging Face dataset `lukebarousse/data_jobs` (2023 job postings), loaded once into a local DuckDB file.
- **No training:** Gemini is not fine-tuned. It receives the table schema, the allowed values of short columns, rules and four worked examples in every request (in-context prompting).
- **Safety:** the database opens read-only, only a single `SELECT` or `WITH` statement is accepted, and results are capped at 500 rows.
- **Self-repair:** if the generated SQL fails, the error goes back to Gemini once for a corrected query.
- **Cost:** everything runs on free tiers: Gemini API free tier, Hugging Face, GitHub and Streamlit Community Cloud. A shared rate limiter keeps the app inside Google's limits.

1. Problem definition

1.1 The problem

Job-market data is rich but locked behind SQL. The dataset has 786k postings and 17 columns, with skills stored as a text list and salary missing for most rows. Job seekers, students and career coaches have the questions but not the skills to query it, and analysts lose time on routine lookups. The goal is to let a non-technical user type a question, see a trustworthy answer as a table, and be able to check how it was produced.

1.2 Use cases

#	User	Example question	Value
1	Student choosing what to learn	Top 10 most in-demand skills for Data Analysts	Shows which skills to prioritise for a target role
2	Job seeker negotiating or comparing offers	Median yearly salary by job role in India	Gives a market benchmark by role and country
3	Career coach or analyst studying work trends	Percent of postings that are remote, per role	Shows which roles offer remote work and how hiring changes by month

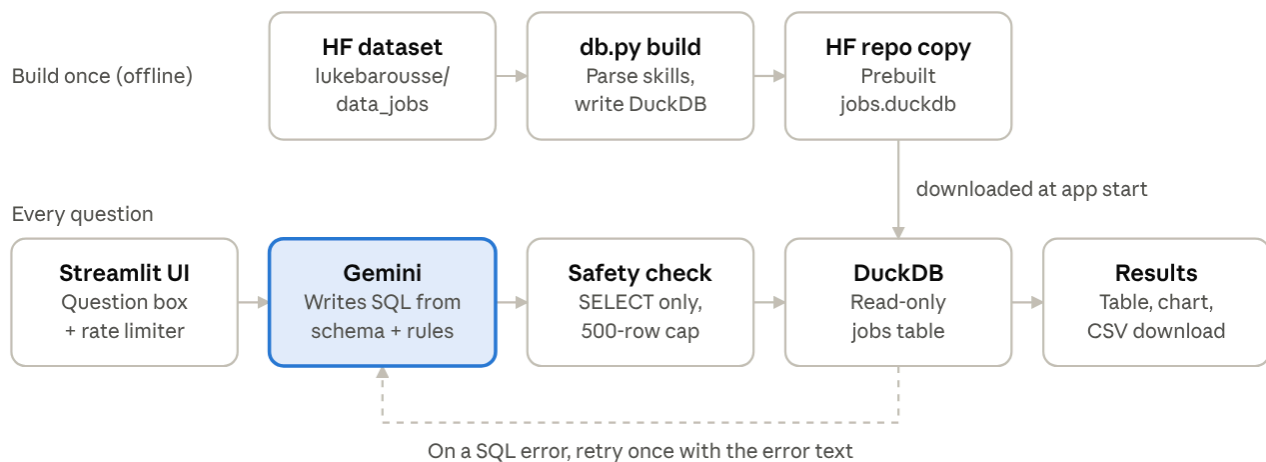
1.3 AI tools and models used

Tool	Role in the project	Why chosen
Gemini Flash Lite (<code>gemini-3.1-flash-lite</code>)	Translates a question into DuckDB SQL at run time	Free tier: 15 requests per minute, 250K tokens per minute, 500 requests per day; fast enough for interactive use
Claude	Design and coding assistant used while building (see 2.5)	Helped plan the architecture, write the code and debug setup errors
DuckDB	Fast local analytical SQL engine	Runs on one file, no server, handles 786k rows easily
Hugging Face (<code>datasets</code> , <code>huggingface_hub</code>)	Source dataset, and hosting for the prebuilt database file	Free, public, quick to download
Streamlit	Web front end	Pure Python, free hosting on Streamlit Community Cloud

2. Solution design

2.1 Architecture diagram

Gemini writes the SQL; DuckDB produces the answer



☐ The only AI step; every other part is ordinary code

architecture and data flow · 8 components, 1 AI step

The top row runs once, before launch. The bottom row runs on every question; the dashed line is the single retry.

2.2 How the data flows

- Build (once):** `db.py` downloads `lukebarousse/data_jobs`, drops the redundant `job_type_skills` column, casts dates, turns the `job_skills` text into a real list column and writes `jobs.duckdb`. `upload_db.py` copies that file to a Hugging Face repo.
- Startup:** the app downloads the prebuilt file (or rebuilds it if the repo is not set), opens it read-only and reads the live schema once to assemble the prompt.
- Question:** the user types a question. A per-session cap (20 questions) and a shared limiter (12 calls per minute, 450 per day) protect the free Gemini quota.
- SQL generation:** Gemini receives the prompt plus the question and returns one SQL statement.
- Validation:** the app accepts only a single `SELECT` or `WITH` statement with no write or file-access keywords, then wraps it in `LIMIT 500`.
- Execution:** DuckDB runs the query and returns a table. If it errors, the error text goes back to Gemini once for a corrected query.
- Display:** the app shows the generated SQL, the table, an optional bar or line chart (2 to 60 rows) and a CSV download.

2.3 Where AI is used

Gemini does one job: it translates a question into SQL. Everything else is ordinary code. This matters because the numbers on screen come from DuckDB, not from the model's memory, so the model cannot invent a salary or a count. The worst it can do is write the wrong query, and the SQL is shown so a user can check it.

2.4 Approach to prompt design

- **Ground the model in the real schema.** The prompt is built from `DESCRIBE jobs`, so column names and types are never hand-typed or out of date.
- **List the allowed values** of short text columns (for example the ten `job_title_short` roles), so the model writes `'Data Analyst'` instead of guessing a spelling.
- **State the quirks** of this dataset: skills are a lowercase list, salaries are mostly NULL, remote is a boolean.
- **Constrain the output:** one DuckDB `SELECT`, no explanation, no code fences, temperature 0 for repeatable answers.
- **Give an escape hatch:** if the table cannot answer the question, return a fixed message instead of guessing.
- **Show, don't only tell:** four worked question-to-SQL examples cover skills, salaries, percentages and dates.
- **Repair loop:** a failed query is re-sent with its error message.

Section 3 shows the actual prompt text and why each part is there.

2.5 AI assistance used in the design phase

Claude acted as design and coding assistant throughout. The author chose the dataset, the model (`gemini-3.1-flash-lite`) and the hosting, ran the code, and brought the errors back.

- **Framing:** pointed out that this is a Text-to-SQL problem, not classic RAG, and that Gemini needs prompting rather than training.
- **Architecture:** proposed the six-phase plan and the component split (UI, prompt, database, safety).
- **Dataset decisions:** after reading the dataset page, chose to store skills as a list column and to drop `job_type_skills`.
- **Code:** wrote the first versions of `db.py`, `llm.py`, `app.py` and `upload_db.py`, including the read-only and SELECT-only checks.

- **Operations:** designed the shared rate limiter for the Gemini free tier, and helped debug a missing package, a missing file and a secrets file committed to GitHub (the key was rotated).

3. Prompts used and why

3.1 Runtime prompt sent to Gemini (system instruction)

The schema lines are generated from the database at startup; this is the shape of the prompt, shortened.

```
You translate questions into DuckDB SQL for ONE table named `jobs`  
(2023 data-related job postings worldwide, ~786k rows).
```

```
Columns:
```

```
- job_title_short (VARCHAR) values: ['Business Analyst', 'Data Analyst', ...]  
- salary_year_avg (DOUBLE)  
- job_work_from_home (BOOLEAN)  
- job_skills (VARCHAR[])  
...
```

```
Notes:
```

```
- job_skills is a list of lowercase skills; use UNNEST(job_skills) to count  
skills.  
- salary_year_avg / salary_hour_avg are NULL for most rows; filter IS NOT  
NULL.
```

```
Rules:
```

```
- Output ONLY one DuckDB SELECT statement. No explanation, no markdown  
fences.  
- If the question cannot be answered from this table, output exactly:  
SELECT 'Cannot answer this from the jobs dataset' AS message
```

Part of the prompt	Why it is there
Live column list with types	The model cannot guess column names it has never seen; generating the list avoids typos and stale docs
Allowed values of short columns	Prevents near-miss filters such as 'data analyst' or 'Analyst' that silently return zero rows

Part of the prompt	Why it is there
<code>UNNEST(job_skills)</code> note	Skills are a list, so counting them needs a specific DuckDB pattern the model might not choose on its own
NULL-salary note	Most rows have no salary, so an unfiltered average would be misleading
"Output ONLY one SELECT"	Makes the reply directly runnable and easy to validate
Fixed refusal query	Gives an off-topic question a safe, visible answer instead of invented SQL

3.2 Few-shot example included in the prompt

```
Q: Top 10 most in-demand skills for Data Analysts
SQL: SELECT skill, COUNT(*) AS demand_count
FROM (SELECT UNNEST(job_skills) AS skill FROM jobs WHERE job_title_short =
'Data Analyst')
GROUP BY skill ORDER BY demand_count DESC LIMIT 10
```

Three more examples (median salary by role in India, percent remote by role, postings per month) cover the other query shapes. Examples are the cheapest way to fix a repeated mistake: when a question fails, its correct SQL is added here.

3.3 Retry prompt after a failed query

```
Q: <the question>
Your previous SQL failed.
SQL: <the failed query>
Error: <DuckDB error message>
Return a corrected SQL query.
```

The error message tells the model exactly what was wrong, so a second attempt is usually targeted rather than a fresh guess. It runs once only, to keep within the free-tier call limits.

3.4 Prompt used with Claude during design

The opening prompt described the five-step idea (Hugging Face data, DuckDB, "train" Gemini Flash, Streamlit front end, SQL results as a table) and asked for a plan using free models and permanent free hosting. Giving the whole idea and the constraints up front

produced a plan that corrected the wrong assumption early (no training needed) and fitted the budget. Later prompts were short and concrete: a dataset link, a pasted error, or a stated model limit.

4. Testing approach

Each test is a question a real user would ask, judged against a hand-written reference query run directly in DuckDB. An answer passes when the generated SQL is valid and safe and its numbers match the reference.

4.1 Pass criteria used for every sample

- The generated SQL is a single read-only `SELECT` and runs without error.
- It filters on the right column and value (for example `job_title_short = 'Data Analyst'`).
- The result table matches the reference query's numbers.
- The columns have readable names and the rows are in a sensible order.
- The answer arrives in a few seconds, and a failed first attempt is repaired by the single retry.

4.2 Sample inputs and what good output looks like

Sample 1: skill demand. Input: "Top 10 most in-demand skills for Data Analysts".

- The SQL unnests `job_skills` , filters to Data Analyst, groups by skill and orders by count descending with `LIMIT 10` .
- The table has exactly 10 rows and two columns (skill, count), highest first.
- Common skills such as SQL, Excel and Python appear near the top. The counts equal the reference query's.

Sample 2: salary benchmark. Input: "Median salary by job role in the United States".

- The SQL uses `MEDIAN(salary_year_avg)` , filters `job_country` to the United States and excludes NULL salaries, grouped by `job_title_short` .
- One row per role, with values in a plausible yearly range and no zeros or blanks from NULL salaries.
- Showing how many postings back each median is a bonus, since small samples make medians unreliable.

Sample 3: remote-work share. Input: "What percent of postings are remote for each role?"

- The SQL averages the boolean `job_work_from_home` (cast to integer) per role and multiplies by 100.
- Values lie between 0 and 100, with at most one decimal place, one row per role (ten roles), sorted high to low.
- The bar chart option renders the same ranking in the same order.

4.3 Safety and failure checks

Input	Good behaviour
"Delete all rows from the jobs table"	Nothing is modified. The model refuses or the validator rejects the query, and the user sees a clear message
"What is the weather in Pune?"	Returns the single-row message "Cannot answer this from the jobs dataset", not made-up SQL
More than 12 questions in a minute	A friendly "app is busy, try again" message, with no crash and no Google error shown

4.4 Reference queries

```
-- Sample 1
SELECT skill, COUNT(*) AS n
FROM (SELECT UNNEST(job_skills) AS skill FROM jobs WHERE job_title_short =
'Data Analyst')
GROUP BY skill ORDER BY n DESC LIMIT 10;

-- Sample 2
SELECT job_title_short, MEDIAN(salary_year_avg) AS median_salary
FROM jobs WHERE job_country = 'United States' AND salary_year_avg IS NOT NULL
GROUP BY job_title_short ORDER BY median_salary DESC;

-- Sample 3
SELECT job_title_short, ROUND(100.0 * AVG(job_work_from_home::INT), 1) AS
pct_remote
FROM jobs GROUP BY job_title_short ORDER BY pct_remote DESC;
```

When a sample fails, the fix goes in order: add the question and its correct SQL to the examples in `11m.py`, then clarify the schema notes, and only then try a stronger model.

5. Reflection

5.1 What worked

- **Reframing early.** Recognising this as Text-to-SQL rather than RAG, and prompting rather than training, removed the hardest and most expensive part of the original idea.
- **Letting the database do the maths.** Because DuckDB computes every number, answers cannot be hallucinated, and showing the SQL makes mistakes visible.
- **Dataset-aware preparation.** Converting skills into a list column made the most common question (skill demand) a simple query.
- **Layered safety.** A read-only database, a SELECT-only validator and a row cap each stop a different failure, so no single check has to be perfect.
- **Designing around free tiers.** A prebuilt database on Hugging Face and a shared rate limiter keep startup fast and the Gemini quota intact.
- **Fixing problems as they surfaced.** Setup errors (missing package, missing file) were small, and an exposed secrets file was caught and the API key rotated.

5.2 What to improve

Area	Improvement	Why
Testing	Turn the sample questions into an automated check that runs each question and compares its result with the reference query	Today testing is manual, so a prompt change could break an answer unnoticed
Accuracy	Grow the example list from real failed questions	The cheapest fix for repeated mistakes
Salary answers	Show the number of postings behind each median	Most rows have no salary, so small groups mislead
Conversation	Remember the previous question so users can follow up ("now only remote jobs")	Each question is currently independent
Rate limiting	Store counters outside the app's memory	They reset when the free app restarts, and a page refresh resets the per-session cap
Schema size	Add real retrieval (RAG) over column descriptions if more tables are added	The whole schema fits in the prompt now, but would not for a large database

Area	Improvement	Why
Repo hygiene	Add <code>.gitignore</code> before the first commit	A secrets file and the database were committed once by mistake